Chapter 10. Attention and Transformers

Lecture 11/2: attention-based captioning, spatial and channel attention, inductive bias, scaled dot-product, cross- and self-attention, positional encoding, masking, multi-head attention, Transformer encoder, layer normalization

Contents

1 Attention-Based Image Captioning **2**

2 Spatial and Channel Attention **3**

3 Inductive Bias **3**

4 The General Attention Layer **4**

4.1 One query 4

4.2 Why divide by $\sqrt{D}$ 5

4.3 Many queries, keys and values 6

5 Permutation Symmetry **7**

6 Positional Encoding **7**

7 Masked Self-Attention **8**

8 Multi-Head Attention **10**

9 The Transformer Encoder **10**

10 Layer Normalization **11**

11 Linearizing Attention **12**

12 Why Transformers **13**

13 Selected exercises **13**

1 Attention-Based Image Captioning

Motivation. Naive captioning (Chapter 9) feeds one global image vector once, and a strong language model learns to ignore it. Attention lets each generated word look at the image regions it describes, by recomputing a weighted summary of the spatial feature map at every step.

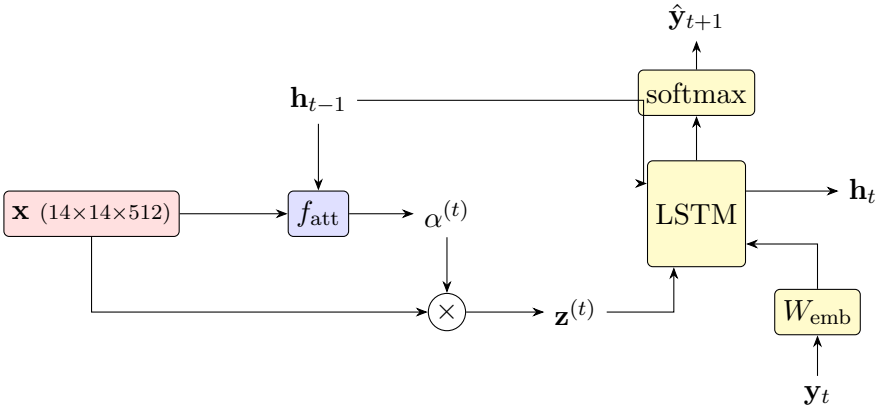
Instead of the fully connected feature vector, keep the last convolutional feature map of the pretrained CNN: for a $224 \times 224 \times 3$ input, conv5 of VGG has shape $14 \times 14 \times 512$. Write it as $L = 14 \cdot 14 = 196$ feature vectors $\mathbf{x}_{i,j} \in \mathbb{R}^{512}$, one per spatial location. An **attention map** for a word is a nonnegative matrix $\alpha^{(t)} \in \mathbb{R}^{14\times 14}$ with the same spatial size as the feature map and $\sum_{i,j} \alpha^{(t)}_{i,j} = 1$. There is one per generated word ($\alpha^{(1)}$ for "a", $\alpha^{(2)}$ for "bird", ...), and it summarizes the image as $\mathbf{z}^{(t)} = \sum_{i,j} \alpha^{(t)}_{i,j} \mathbf{x}_{i,j} \in \mathbb{R}^{512}$. In sizes: $\mathbf{x} \in \mathbb{R}^{14\times 14\times 512}$, $\mathbf{x}_{i,j} \in \mathbb{R}^{512}$, $\alpha^{(t)} \in \mathbb{R}^{14\times 14}$. Two questions follow: how to compute the attention, and how to use it to predict the word.

Definition 10.1: attention-based captioning (Show, Attend and Tell)

At step t, with previous hidden state $\mathbf{h}_{t-1}$ and current input word $\mathbf{y}_t$:

$$
\begin{aligned}
e^{(t)} &= f_{\text{att}}(\mathbf{x}, \mathbf{h}_{t-1}) && \text{(alignment scores, one per location)},\\
\alpha^{(t)}_{i,j} &= \frac{\exp(e^{(t)}_{i,j})}{\sum_{i',j'} \exp(e^{(t)}_{i',j'})} && \text{(attention weights)},\\
\mathbf{z}^{(t)} &= \sum_{i,j} \alpha^{(t)}_{i,j} \mathbf{x}_{i,j} && \text{(context vector)},\\
\mathbf{h}_t &= \text{LSTM}(\mathbf{h}_{t-1}, W_{\text{emb}}\mathbf{y}_t, \mathbf{z}^{(t)}) && \text{(modified LSTM)},\\
\hat{\mathbf{y}}_{t+1} &\propto \exp\left(W^o(W_{\text{emb}}\mathbf{y}_t + W^h\mathbf{h}_t + W^z\mathbf{z}^{(t)})\right) && \text{(next-word distribution)}.
\end{aligned}
$$

The modified LSTM takes the context vector as an extra input to every gate. The attention module f_{att} is a small MLP; a common choice is a one-hidden-layer network, $e_{i,j} = \mathbf{w}^\top \tanh(W_x\mathbf{x}_{i,j} + W_h\mathbf{h}_{t-1})$ [Supplement]. The cell is repeated N times, once per generated word.



One step of attention-based captioning: the attention module turns $(\mathbf{x}, \mathbf{h}_{t-1})$ into weights $\alpha^{(t)}$, and the weighted feature $\mathbf{z}^{(t)}$ enters the LSTM.

Theorem 10.2: the context vector is a convex combination [Supplement]

Each $\alpha^{(t)}$ is a probability vector over the L locations, so $\mathbf{z}^{(t)}$ lies in the convex hull of $\{\mathbf{x}_{i,j}\}$. With uniform scores ($e^{(t)}$ constant) it equals the global average-pooled feature $\frac{1}{L}\sum_{i,j} \mathbf{x}_{i,j}$.

Proof. Step 1. Each $\alpha_{i,j}^{(t)}$ is positive, and the weights sum to 1 because the denominator is the sum of the numerators. A sum $\sum \alpha_{i,j}\mathbf{x}_{i,j}$ with such weights is by definition a point of the convex hull.

Step 2. If $e_{i,j}^{(t)} = c$ for all locations, every weight equals $e^c/(Le^c) = 1/L$. ■

So "no attention" corresponds to uniform attention, which reduces to the naive model with a pooled feature.

Soft versus hard attention. The rule above is **soft attention**: a deterministic weighted average, differentiable in all parameters and trained by backpropagation. **Hard attention** instead samples one location $s_t \sim \text{Categorical}(\alpha^{(t)})$ and sets $\mathbf{z}^{(t)} = \mathbf{x}_{s_t}$. In the lecture's visualizations, soft attention spreads over the bird and the water, while hard attention picks one spot per word ("bird", "water").

Theorem 10.3: soft attention is the mean of hard attention [Supplement]

Under hard attention, $\mathbb{E}[\mathbf{z}^{(t)} \mid \mathbf{x}, \mathbf{h}_{t-1}]$ equals the soft-attention context vector.

Proof. Given $\mathbf{x}$ and $\mathbf{h}_{t-1}$, $\mathbf{z}^{(t)} = \mathbf{x}_{s_t}$ takes the value $\mathbf{x}_{i,j}$ with probability $\alpha_{i,j}^{(t)}$. Hence $\mathbb{E}[\mathbf{z}^{(t)}] = \sum_{i,j} \alpha_{i,j}^{(t)}\mathbf{x}_{i,j}$. ■

The sampling step of hard attention has no gradient with respect to α, so hard attention is trained with score-function (REINFORCE-type) gradient estimators rather than plain backpropagation [Supplement].

2 Spatial and Channel Attention

Attention weights can be placed on different axes of a feature map $U \in \mathbb{R}^{H\times W\times C}$.

- **Spatial attention** weights the $H \times W$ locations (Section 1).
- **Channel attention** weights the C channels.

Definition 10.4: squeeze-and-excitation block (SENet)

Given $U = [\mathbf{u}_1, \ldots, \mathbf{u}_C]$ with $\mathbf{u}_c \in \mathbb{R}^{H\times W}$:

$$
\begin{aligned}
s_c &= \frac{1}{HW}\sum_{i=1}^{H}\sum_{j=1}^{W} u_c(i,j) && \text{(squeeze: global average pooling)},\\
\mathbf{e} &= \sigma\big(W_2\ \mathrm{ReLU}(W_1\mathbf{s})\big) && \text{(excitation)},\\
\tilde{\mathbf{u}}_c &= e_c\,\mathbf{u}_c && \text{(scale)},
\end{aligned}
$$

with $W_1 \in \mathbb{R}^{(C/r)\times C}$ and $W_2 \in \mathbb{R}^{C\times(C/r)}$. In words: global pooling, then a channel-wise weight, then multiplication with the original map.

Each $e_c \in (0, 1)$ comes from a sigmoid, not a softmax, so here the attention weights need not sum to 1. The block adds $2C^2/r$ weights (reduction ratio r, e.g. 16) and is inserted into residual blocks (SE-ResNet) or mobile blocks (ANTNets).

3 Inductive Bias

Motivation. We now have MLPs, CNNs, RNNs and attention. Why different models for different data? Each architecture builds in assumptions about its data, and attention is the one with the

fewest.

So far we have seen four families:

- MLP: linear layers and nonlinear functions.
- CNN: convolution, pooling, nonlinearities and an MLP head.
- RNN: MLPs for the recurrent update, plus gating functions.
- Attention and Transformers: dot-product attention plus MLPs.

One can also mix data and models: a CNN for sequences (one-dimensional convolution), an RNN for images (scanning pixels in order), an MLP for images (flattened pixels).

Definition 10.5: inductive bias

The **inductive bias** of a learner is the set of assumptions it uses to predict outputs for inputs it has not observed.

Most architectural biases concern **locality** and **invariance**.

- Convolution exploits the 2D grid structure and translation equivariance (Chapter 7).
- An RNN exploits a 1D chain and a first-order Markov assumption: $\mathbf{h}_t$ depends on the past only through $\mathbf{h}_{t-1}$.

These assumptions make models efficient, but they often do not transfer across data modalities. The goal is one model for images, point clouds, audio and language. The key requirement is to make few assumptions about the input structure.

Remark 10.6: an MLP has no spatial bias [Supplement]

Let P be a fixed permutation matrix of the n input pixels. A first layer $\mathbf{x} \mapsto W\mathbf{x}$ on the shuffled images $P\mathbf{x}$ realizes exactly the same functions as on the originals, since $W(P\mathbf{x}) = (WP)\mathbf{x}$ and $W \mapsto WP$ is a bijection of $\mathbb{R}^{m\times n}$. So an MLP learns shuffled images as well as natural ones. A convolution is not closed under this change, because WP is in general not a convolution matrix.

4 The General Attention Layer

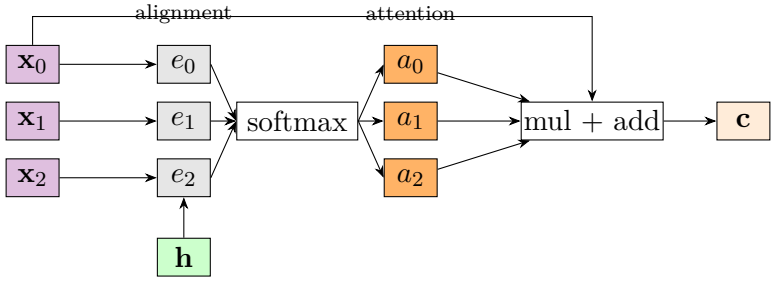
4.1 One query

Definition 10.7: attention with one query

Inputs: vectors $\mathbf{x}_1, \dots, \mathbf{x}_N \in \mathbb{R}^D$ (stacked as $\mathbf{x} \in \mathbb{R}^{N\times D}$) and a query $\mathbf{h} \in \mathbb{R}^D$.

$$\begin{aligned} \text{alignment:} \quad & e_i = f_{\text{att}}(\mathbf{h}, \mathbf{x}_i) = \frac{\mathbf{h}\cdot\mathbf{x}_i}{\sqrt{D}}, \\ \text{attention:} \quad & \mathbf{a} = \operatorname{softmax}(\mathbf{e}), \\ \text{output:} \quad & \mathbf{c} = \sum_{i=1}^{N} a_i \mathbf{x}_i \in \mathbb{R}^D \quad \text{(context vector)}. \end{aligned}$$

A feature map of size $H \times W \times D$ is stretched into $N = HW$ vectors. The operation does not care about the order of the features: reordering $\mathbf{x}_1, \ldots, \mathbf{x}_N$ leaves $\mathbf{c}$ unchanged (Theorem 10.13).



The general attention layer with one query $\mathbf{h}$*: scores, softmax, then a weighted sum of the inputs.*

4.2 Why divide by $\sqrt{D}$

The lecture replaced a general f_{att} by a *scaled* dot product. The reasoning: in higher dimension the dot product has more terms, so the logits have larger variance. Large logits give a peaked, low-entropy softmax that puts almost all weight on one input. Dividing by $\sqrt{D}$ removes the growth. Two results make this precise.

Theorem 10.8: variance of a dot product [Supplement]

Let $\mathbf{h}$ and $\mathbf{x}$ be independent random vectors in $\mathbb{R}^D$ whose $2D$ entries are independent with mean 0 and variance 1. Then $\mathbb{E}[\mathbf{h} \cdot \mathbf{x}] = 0$ and $\operatorname{Var}(\mathbf{h} \cdot \mathbf{x}) = D$. Hence $\mathbf{h} \cdot \mathbf{x}/\sqrt{D}$ has variance 1 for every D.

Proof. Step 1. By independence, $\mathbb{E}[h_k x_k] = \mathbb{E}[h_k]\mathbb{E}[x_k] = 0$, so $\mathbb{E}[\mathbf{h} \cdot \mathbf{x}] = 0$.

Step 2. For $k \neq l$, $\mathbb{E}[h_k x_k h_l x_l] = \mathbb{E}[h_k]\mathbb{E}[x_k]\mathbb{E}[h_l]\mathbb{E}[x_l] = 0$, so the cross terms of $(\sum_k h_k x_k)^2$ have mean zero. For $k = l$, $\mathbb{E}[h_k^2 x_k^2] = \mathbb{E}[h_k^2]\mathbb{E}[x_k^2] = 1$. Hence $\operatorname{Var}(\mathbf{h} \cdot \mathbf{x}) = \mathbb{E}[(\mathbf{h} \cdot \mathbf{x})^2] = D$.

Step 3. Scaling by $1/\sqrt{D}$ divides the variance by D. ■

Theorem 10.9: a saturated softmax passes almost no gradient [Supplement]

Let $\mathbf{a} = \operatorname{softmax}(\mathbf{e}) \in \mathbb{R}^N$. Then:

(a) The Jacobian is $J = \dfrac{\partial \mathbf{a}}{\partial \mathbf{e}} = \operatorname{diag}(\mathbf{a}) - \mathbf{a}\mathbf{a}^\top$.

(b) J is symmetric positive semidefinite and $J\mathbf{1} = \mathbf{0}$.

(c) If $a_k \geq 1 - \varepsilon$ for some k, then $\|J\|_2 \leq 2\varepsilon$.

Proof. (a) Write $a_i = e^{e_i}/S$ with $S = \sum_j e^{e_j}$. Then $\partial a_i / \partial e_j = \delta_{ij} e^{e_i}/S - e^{e_i} e^{e_j}/S^2 = a_i(\delta_{ij} - a_j)$.

(b) Symmetry is clear. For $\mathbf{v} \in \mathbb{R}^N$,

$$\mathbf{v}^\top J \mathbf{v} = \sum_i a_i v_i^2 - \Big(\sum_i a_i v_i\Big)^2,$$

which is the variance of v_I when the index I has distribution $\mathbf{a}$, hence ≥ 0. Also $(J\mathbf{1})_i = a_i - a_i \sum_j a_j = 0$.

(c) A positive semidefinite matrix has spectral norm at most its trace (its largest eigenvalue is at most the sum of all eigenvalues). Here $\operatorname{tr} J = \sum_i a_i(1 - a_i) \leq (1 - a_k) + \sum_{i \neq k} a_i = 2(1 - a_k) \leq 2\varepsilon$. ■

Without scaling, Theorem 10.8 gives logits of standard deviation $\sqrt{D}$. For large D the largest logit is then typically far above the others, the softmax is nearly one-hot, and by (c) the gradient through the attention weights nearly vanishes. Exercise 10.1 shows the same effect through the entropy.

4.3 Many queries, keys and values

Definition 10.10: multiple queries

With queries $\mathbf{q}_1, \dots, \mathbf{q}_M \in \mathbb{R}^D$ (stacked as $\mathbf{q} \in \mathbb{R}^{M \times D}$), each query produces its own context vector:

$$e_{i,j} = \frac{\mathbf{q}_j \cdot \mathbf{x}_i}{\sqrt{D}}, \qquad \mathbf{a}_{\cdot,j} = \operatorname{softmax}(\mathbf{e}_{\cdot,j}), \qquad \mathbf{y}_j = \sum_i a_{i,j} \mathbf{x}_i.$$

Here the inputs $\mathbf{x}$ play two roles, in the alignment and in the output. Adding a separate fully connected layer before each role makes the layer more expressive, and lets the input and output dimensions change.

Definition 10.11: cross-attention layer

With learnable $W_k \in \mathbb{R}^{D \times D_k}$ and $W_v \in \mathbb{R}^{D \times D_v}$ and queries $\mathbf{q} \in \mathbb{R}^{M \times D_k}$:

$$\mathbf{k} = \mathbf{x}W_k, \quad \mathbf{v} = \mathbf{x}W_v, \quad e_{i,j} = \frac{\mathbf{q}_j \cdot \mathbf{k}_i}{\sqrt{D_k}}, \quad \mathbf{a}_{\cdot,j} = \operatorname{softmax}(\mathbf{e}_{\cdot,j}), \quad \mathbf{y}_j = \sum_i a_{i,j} \mathbf{v}_i \in \mathbb{R}^{D_v}.$$

Definition 10.12: self-attention layer

If the queries are also computed from the inputs, $\mathbf{q} = \mathbf{x}W_q$ with $W_q \in \mathbb{R}^{D \times D_k}$, the layer maps N inputs to N outputs and is called **self-attention**. In matrix form, with rows as tokens,

$$\mathrm{SA}(\mathbf{x}) = \mathrm{Attn}(\mathbf{x}W_q, \mathbf{x}W_k, \mathbf{x}W_v) = \operatorname{softmax}\Big(\frac{\mathbf{x}W_q(\mathbf{x}W_k)^\top}{\sqrt{D_k}}\Big)\mathbf{x}W_v,$$

where the softmax acts on each row. This is the **scaled dot-product attention** of the Transformer, $\mathrm{Attn}(Q, K, V) = \operatorname{softmax}(QK^\top/\sqrt{d_k})V$.

The matrix $A = \operatorname{softmax}(QK^\top/\sqrt{d_k}) \in \mathbb{R}^{n \times n}$ is row-stochastic, and row i of AV is a weighted average of all value vectors. So each output is a contextual encoding of the input whose receptive field is the whole sequence. This is the sense in which attention is **feature aggregation**.

The four steps in the lecture. With $\mathbf{x} \in \mathbb{R}^{n \times d}$ and projection matrices $W_q, W_k \in \mathbb{R}^{d \times d_k}$, $W_v \in \mathbb{R}^{d \times d_v}$, set $Q = \mathbf{x}W_q$ (the **query**, "the question"), $K = \mathbf{x}W_k$ (the **key**, "the index of the information") and $V = \mathbf{x}W_v$ (the **value**, "the information itself"). Then:

(i) $A = QK^\top \in \mathbb{R}^{n \times n}$, where $A_{ij} = \mathbf{q}_i \cdot \mathbf{k}_j$ measures how much token i takes the information of token j into account;

(ii) $A' = QK^\top/\sqrt{d_k}$ (scaling): if the entries of $\mathbf{q}, \mathbf{k}$ are independent with mean 0 and variance 1, then $z = \sum_{m=1}^{d_k} q_m k_m$ has $\mathbb{E}z = 0$ and $\operatorname{Var} z = \sum_m \operatorname{Var}(q_m k_m) = d_k$, so dividing by $\sqrt{d_k}$ restores unit variance;

(iii) $\alpha_{ij} = \operatorname{softmax}(A')_{ij}$, row by row;

(iv) $\text{output}_i = \sum_j \alpha_{ij} \mathbf{v}_j$.

Cross versus self. In cross-attention the queries come from elsewhere (e.g. a decoder) and the keys and values from the input X; in self-attention Q, K and V all come from X. Attention-based captioning (Section 1) is cross-attention: the query is the state of the sentence being generated, and the keys and values are the image features. In the single-query layer of Definition 10.7 the keys and values coincide (both are $\mathbf{x}_i$).

5 Permutation Symmetry

Theorem 10.13: permutation invariance and equivariance [Supplement]

Let $P \in \mathbb{R}^{N\times N}$ be a permutation matrix.

(a) For a row-wise softmax and any $L \in \mathbb{R}^{N\times N}$, $\operatorname{softmax}(PLP^\top) = P\operatorname{softmax}(L)\,P^\top$, and $\operatorname{softmax}(LP^\top) = \operatorname{softmax}(L)P^\top$.

(b) Cross-attention with fixed queries is **permutation invariant** in its inputs: replacing $\mathbf{x}$ by $P\mathbf{x}$ leaves the outputs $\mathbf{y}$ unchanged.

(c) Self-attention is **permutation equivariant**: $\mathrm{SA}(P\mathbf{x}) = P\,\mathrm{SA}(\mathbf{x})$.

Proof. (a) Let P send row i to row $\pi(i)$. A row-wise softmax treats rows separately, so permuting rows before or after it gives the same result. Within one row, softmax of a vector with permuted entries is the softmax with permuted entries, since both the numerator e^{ℓ_j} and the common denominator $\sum_j e^{\ell_j}$ are unchanged by reordering. Right multiplication by $P^\top$ permutes columns and left multiplication by P permutes rows, so both identities follow.

(b) Stack the queries as rows of $Q \in \mathbb{R}^{M\times D_k}$, so the outputs are $\operatorname{softmax}(Q(\mathbf{x}W_k)^\top/\sqrt{D_k})\,\mathbf{x}W_v$. With $P\mathbf{x}$ in place of $\mathbf{x}$ the logits become $QW_k^\top\mathbf{x}^\top P^\top/\sqrt{D_k} = LP^\top$, so by (a) the output is $\operatorname{softmax}(L)P^\top P\mathbf{x}W_v = \operatorname{softmax}(L)\mathbf{x}W_v$, because $P^\top P = I$.

(c) The logits become $P\mathbf{x}W_qW_k^\top\mathbf{x}^\top P^\top/\sqrt{D_k} = PLP^\top$. By (a),

$$\mathrm{SA}(P\mathbf{x}) = P\operatorname{softmax}(L)P^\top P\mathbf{x}W_v = P\operatorname{softmax}(L)\mathbf{x}W_v = P\,\mathrm{SA}(\mathbf{x}).$$

■

So self-attention alone treats a sentence as a set of words: "dog bites man" and "man bites dog" give the same outputs up to reordering. Word order has to be supplied separately.

6 Positional Encoding

Motivation. By Theorem 10.13, self-attention cannot see positions. We give each token a vector that encodes its position and attach it to the token's embedding.

Use a function $\mathrm{pos} : \mathbb{N} \to \mathbb{R}^d$ that turns position j into a d-dimensional vector $\mathbf{p}_j = \mathrm{pos}(j)$, and attach it to the input $\mathbf{x}_j$. The slides concatenate $\mathbf{p}_j$ to $\mathbf{x}_j$; the Transformer adds it, $\mathbf{x}_j + \mathbf{p}_j$. Two options for pos:

1. **Learn a lookup table**: a parameter vector $\mathrm{pos}(t)$ for each $t \in [0, T)$, so $T \times d$ parameters.
2. **Design a fixed function** with the desired properties.

The intuition for the fixed design is a binary counter. Writing $0, 1, 2, \ldots, 15$ in binary, the lowest bit alternates with period 2, the next with period 4, and so on. Each bit is a periodic function of the position, at a different frequency.

Definition 10.14: sinusoidal positional encoding

For even d and $k = 0, 1, \ldots, \frac{d}{2} - 1$, let $\omega_k = 1/10000^{2k/d}$ and

$$\mathrm{PE}(t)_{2k} = \sin(\omega_k t), \qquad \mathrm{PE}(t)_{2k+1} = \cos(\omega_k t).$$

Equivalently, $\mathrm{PE}(t) = \big(\sin\omega_0 t, \cos\omega_0 t, \ \ldots, \ \sin\omega_{d/2-1} t, \cos\omega_{d/2-1} t\big)$.

Any periodic functions can be used, or the encoding can be learned. The sinusoidal choice has two properties that explain why it is used.

Theorem 10.15: relative positions are linear [Supplement]

(a) For every offset m there is a matrix M_m, not depending on t, with $\mathrm{PE}(t+m) = M_m\,\mathrm{PE}(t)$ for all t. It is block diagonal with 2×2 rotation blocks.

(b) $\mathrm{PE}(t) \cdot \mathrm{PE}(s) = \sum_{k=0}^{d/2-1} \cos\left(\omega_k (t-s)\right)$ depends only on $t - s$. In particular $\|\mathrm{PE}(t)\|^2 = d/2$.

Proof. (a) By the addition formulas,

$$\begin{pmatrix} \sin\omega_k(t+m) \\ \cos\omega_k(t+m) \end{pmatrix} = \begin{pmatrix} \cos\omega_k m & \sin\omega_k m \\ -\sin\omega_k m & \cos\omega_k m \end{pmatrix} \begin{pmatrix} \sin\omega_k t \\ \cos\omega_k t \end{pmatrix}.$$

The 2×2 matrix is a rotation and depends on m but not on t. Stack these blocks along the diagonal for $k = 0, \ldots, \frac{d}{2} - 1$ to get M_m.

(b) The k-th pair contributes $\sin\omega_k t \sin\omega_k s + \cos\omega_k t \cos\omega_k s = \cos(\omega_k(t-s))$. Setting $s = t$ gives $\sum_k 1 = d/2$. ■

By (a), a linear layer can map the encoding of position t to that of $t+m$ in the same way for every t, so attending by relative offset is easy to learn. The frequencies decrease geometrically, so the wavelengths range from 2π up to about $10000 \cdot 2\pi$, like the bits of a counter (Exercise 10.4).

Example 10.16: $d = 4$ (lecture example)

With $d = 4$ there are two frequencies, $\omega_0 = 1$ and $\omega_1 = 1/10000^{2/4} = 0.01$, so $\mathrm{PE}(t) = (\sin t, \cos t, \sin 0.01t, \cos 0.01t)$ and

$$\mathrm{PE}(0) = (0, 1, 0, 1), \qquad \mathrm{PE}(1) \approx (0.841, 0.540, 0.010, 1.000),$$
$$\mathrm{PE}(2) \approx (0.909, -0.416, 0.020, 1.000).$$

The first pair changes quickly with t and the second slowly, like the low and high bits of a binary counter. With the encoding added, $\mathbf{z}_j = \mathbf{x}_j + \mathbf{p}_j$, a token keeps its embedding but shifts by its position: for $\mathbf{x}_{\text{dog}} = (1, 0, 2, 0)$,

$$\mathbf{x}_{\text{dog}} + \mathrm{PE}(0) = (1, 1, 2, 1), \qquad \mathbf{x}_{\text{dog}} + \mathrm{PE}(2) \approx (1.909, -0.416, 2.020, 1.000),$$

so "dog" at position 0 and at position 2 are different inputs, and "dog bites man" is no longer a reordering of "man bites dog".

7 Masked Self-Attention

When generating text, the model must not see future words. The lecture's example: training on "man plays guitar", the Transformer reads the whole shifted sentence at once and predicts every

next token in parallel.

position	input	target
0	<SOS>	man
1	man	plays
2	plays	guitar
3	guitar	<EOS>

At position 1 the target is "plays"; if position 1 could attend to position 2, whose input is "plays", it would copy the answer. So a query at position j may only attend to positions $0, \dots, j$. In self-attention, set the alignment scores from future positions to $-\infty$ by hand. Since $e^{-\infty} = 0$, those positions get zero weight.

Definition 10.17: masked (causal) self-attention

With $\mathbf{q}_i, \mathbf{k}_j, \mathbf{v}_j$ as in Definition 10.12, query i uses the scores

$$e_{i,j} = \begin{cases} \mathbf{q}_i \cdot \mathbf{k}_j / \sqrt{D_k}, & j \le i, \\ -\infty, & j > i, \end{cases} \qquad \mathbf{y}_i = \sum_j a_{i,j} \mathbf{v}_j, \quad \mathbf{a}_{i,\cdot} = \operatorname{softmax}(\mathbf{e}_{i,\cdot}).$$

The slides index the same matrix the other way round, with the key index first: $e_{i,j} = \mathbf{q}_j \cdot \mathbf{k}_i / \sqrt{D_k}$ for $i \le j$ and $-\infty$ for $i > j$, so for three tokens

$$E = \begin{pmatrix} e_{0,0} & e_{0,1} & e_{0,2} \\ -\infty & e_{1,1} & e_{1,2} \\ -\infty & -\infty & e_{2,2} \end{pmatrix}$$

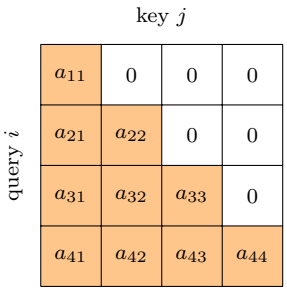
(column j holds the scores of query j), the transpose of the lower-triangular pattern below.

Theorem 10.18: masked attention is causal [Supplement]

The attention matrix $A = (a_{i,j})$ is lower triangular and row-stochastic, and $\mathbf{y}_i$ depends only on $\mathbf{x}_1, \dots, \mathbf{x}_i$.

Proof. Step 1. For $j > i$, $a_{i,j} = e^{-\infty}/(\cdots) = 0$. Row i sums to 1 over $j \le i$, and the denominator is positive because the $j = i$ term is finite.

Step 2. Hence $\mathbf{y}_i = \sum_{j \le i} a_{i,j} \mathbf{v}_j$. Here $\mathbf{v}_j = \mathbf{x}_j W_v$, and $a_{i,j}$ is a function of $\mathbf{q}_i = \mathbf{x}_i W_q$ and of $\mathbf{k}_1, \dots, \mathbf{k}_i$, where $\mathbf{k}_l = \mathbf{x}_l W_k$. So only $\mathbf{x}_1, \dots, \mathbf{x}_i$ enter. ■



A causal attention matrix for four tokens: each query sees only itself and earlier tokens.

8 Multi-Head Attention

Definition 10.19: multi-head attention

With h heads, each with its own projections $W_q^{(r)}, W_k^{(r)} \in \mathbb{R}^{d\times d_k}$ and $W_v^{(r)} \in \mathbb{R}^{d\times d_v}$,

$$\text{head}_r = \text{Attn}\left(QW_q^{(r)}, KW_k^{(r)}, VW_v^{(r)}\right), \qquad \text{MHA}(Q,K,V) = [\text{head}_1, \ldots, \text{head}_h]\, W_o,$$

with $W_o \in \mathbb{R}^{hd_v\times d}$. Usually $d_k = d_v = d/h$.

Each head runs scaled dot-product attention (MatMul, Scale, optional Mask, SoftMax, MatMul) on its own linear projections, and the results are concatenated and mixed by a final linear layer. Different heads can attend to different relations, for example one to the previous word and one to the subject of the sentence.

Theorem 10.20: heads add up [Supplement]

Split W_o into row blocks $W_o^{(1)}, \ldots, W_o^{(h)} \in \mathbb{R}^{d_v\times d}$. Then

$$\text{MHA}(Q,K,V) = \sum_{r=1}^{h} \text{head}_r\, W_o^{(r)}.$$

With $d_k = d_v = d/h$ the layer has $4d^2$ weights for every h.

Proof. Step 1. Block multiplication: $[B_1, \ldots, B_h]\begin{pmatrix} C_1 \\ \vdots \\ C_h \end{pmatrix} = \sum_r B_r C_r$.

Step 2. The projections contribute $3 \cdot h \cdot d \cdot \frac{d}{h} = 3d^2$ weights, and W_o contributes $h\frac{d}{h} \cdot d = d^2$. ■

9 The Transformer Encoder

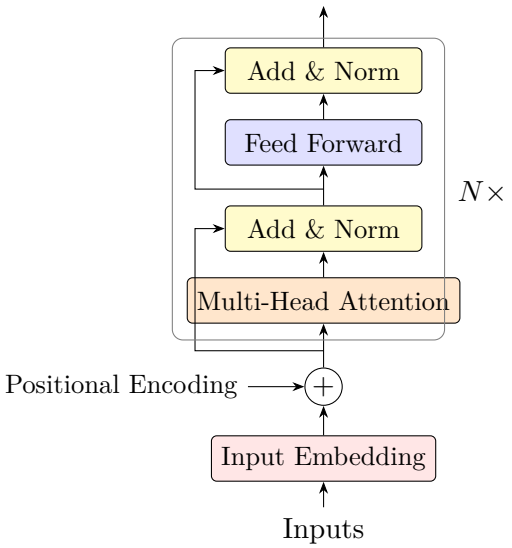
Motivation. The Transformer (Vaswani et al., 2017) stacks self-attention and position-wise MLPs, each with a skip connection and layer normalization.

Definition 10.21: Transformer encoder

1. The input is tokenized, $\mathbf{x} \in \mathbb{R}^{n\times d}$ after each token is encoded into a continuous embedding.
2. A positional encoding is added: $X_0 = \mathbf{x} + \text{PE}$.
3. Each of N layers applies multi-head self-attention with a skip connection, then a feed-forward network with a skip connection:

$$Z = \text{LN}\left(X + \text{MHA}(X,X,X)\right), \qquad X' = \text{LN}\left(Z + \text{FFN}(Z)\right),$$

   where $\text{FFN}(\mathbf{z}) = W_2\,\text{ReLU}(W_1\mathbf{z} + \mathbf{b}_1) + \mathbf{b}_2$ is applied to each token separately.
4. The output of one layer is the input of the next.



One encoder layer (inside the box), repeated N times; the side arrows are the skip connections.

Theorem 10.22: the encoder without positions is permutation equivariant [Supplement]

If the positional encoding is omitted, every encoder layer F satisfies $F(PX) = P\,F(X)$ for every permutation matrix P.

Proof. Step 1. Multi-head self-attention is equivariant: each head is by Theorem 10.13(c), concatenation along columns commutes with a row permutation, and right multiplication by W_o does too.

Step 2. A map applied to each row separately (FFN, layer normalization) is equivariant, since permuting rows just permutes which row each output belongs to. The skip connections add equivariant maps.

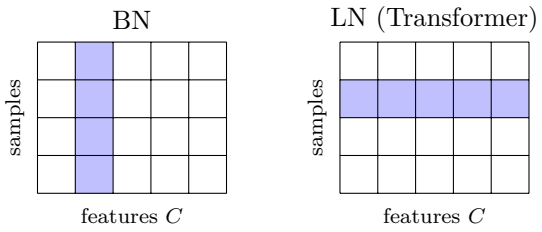
Step 3. Sums and compositions of equivariant maps are equivariant. ■

Thus positions enter only through PE. The cost of a layer is dominated by $QK^\top \in \mathbb{R}^{n\times n}$: $O(n^2 d)$ time and $O(n^2)$ memory in the sequence length n.

10 Layer Normalization

Batch normalization (Chapter 6) normalizes each channel using statistics over the mini-batch. Layer normalization normalizes each sample by itself.

- **BN**: for each channel C, average over the batch N and the positions H, W.
- **LN in CNNs**: for each sample, average over C, H, W.
- **LN in Transformers**: for each sample and each token, average over the C features only.



The shaded cells share one mean and variance: one feature over all samples (BN) versus all features of one sample (LN).

Definition 10.23: layer normalization

For $\mathbf{x} \in \mathbb{R}^d$, let $\mu = \frac{1}{d}\sum_k x_k$ and $\sigma^2 = \frac{1}{d}\sum_k (x_k - \mu)^2$. Then

$$\hat{\mathbf{x}} = \frac{\mathbf{x} - \mu\mathbf{1}}{\sqrt{\sigma^2 + \epsilon}}, \qquad \mathrm{LN}(\mathbf{x}) = \boldsymbol{\gamma} \odot \hat{\mathbf{x}} + \boldsymbol{\beta},$$

with learnable $\boldsymbol{\gamma}, \boldsymbol{\beta} \in \mathbb{R}^d$ and a small $\epsilon > 0$.

Theorem 10.24: properties of layer normalization [Supplement]

Take $\epsilon = 0$ and $\sigma > 0$.

(a) $\sum_k \hat{x}_k = 0$ and $\|\hat{\mathbf{x}}\|^2 = d$.

(b) $\mathrm{LN}(a\mathbf{x} + b\mathbf{1}) = \mathrm{LN}(\mathbf{x})$ for all $a > 0$ and $b \in \mathbb{R}$.

(c) LN uses no batch statistics, so it computes the same function at training and test time.

Proof. (a) $\sum_k (x_k - \mu) = d\mu - d\mu = 0$, and $\|\mathbf{x} - \mu\mathbf{1}\|^2 = d\sigma^2$, so $\|\hat{\mathbf{x}}\|^2 = d\sigma^2/\sigma^2 = d$.

(b) The new mean is $a\mu + b$ and the new standard deviation is $a\sigma$. So the centred vector is $a(\mathbf{x} - \mu\mathbf{1})$, and dividing by $a\sigma$ gives the same $\hat{\mathbf{x}}$.

(c) μ and σ are computed from the single vector $\mathbf{x}$. Compare BN, which uses batch statistics in training and running averages at test time (Chapter 6). ■

11 Linearizing Attention

The slide titled "Linearization of TF" was left blank. This section is a [Supplement] account of the standard idea, which removes the $O(n^2)$ cost and connects attention back to recurrent networks.

Write one row of softmax attention as a normalized kernel average,

$$\mathbf{y}_i = \frac{\sum_j \kappa(\mathbf{q}_i, \mathbf{k}_j)\mathbf{v}_j}{\sum_j \kappa(\mathbf{q}_i, \mathbf{k}_j)}, \qquad \kappa(\mathbf{q}, \mathbf{k}) = \exp\left(\mathbf{q} \cdot \mathbf{k}/\sqrt{d_k}\right).$$

Definition 10.25: linear attention [Supplement]

Replace κ by $\kappa(\mathbf{q}, \mathbf{k}) = \phi(\mathbf{q})^\top \phi(\mathbf{k})$ for a feature map $\phi : \mathbb{R}^{d_k} \to \mathbb{R}^r_{>0}$ (e.g. $\phi(\mathbf{u}) = \mathrm{elu}(\mathbf{u}) + 1$ entrywise).

Theorem 10.26: linear attention in linear time [Supplement]

(a) With $S = \sum_j \phi(\mathbf{k}_j)\mathbf{v}_j^\top \in \mathbb{R}^{r\times d_v}$ and $\mathbf{s} = \sum_j \phi(\mathbf{k}_j) \in \mathbb{R}^r$, $\mathbf{y}_i^\top = \phi(\mathbf{q}_i)^\top S \,/\, \phi(\mathbf{q}_i)^\top \mathbf{s}$. All outputs cost $O(nrd_v)$ instead of $O(n^2 d_v)$.

(b) In the causal version ($j \le i$), $S_i = S_{i-1} + \phi(\mathbf{k}_i)\mathbf{v}_i^\top$ and $\mathbf{s}_i = \mathbf{s}_{i-1} + \phi(\mathbf{k}_i)$, and $\mathbf{y}_i^\top = \phi(\mathbf{q}_i)^\top S_i / \phi(\mathbf{q}_i)^\top \mathbf{s}_i$. So causal linear attention is an RNN with a matrix-valued hidden state.

(c) (Random features.) If $\mathbf{w} \sim N(\mathbf{0}, I)$ and $\phi_{\mathbf{w}}(\mathbf{u}) = \exp(\mathbf{w}\cdot\mathbf{u} - \frac{1}{2}\|\mathbf{u}\|^2)$, then $\mathbb{E}_{\mathbf{w}}[\phi_{\mathbf{w}}(\mathbf{q})\phi_{\mathbf{w}}(\mathbf{k})] = \exp(\mathbf{q}\cdot\mathbf{k})$. So the softmax kernel itself is an average of products of positive features.

Proof. (a) By associativity, $\sum_j (\phi(\mathbf{q}_i)^\top \phi(\mathbf{k}_j))\mathbf{v}_j^\top = \phi(\mathbf{q}_i)^\top \sum_j \phi(\mathbf{k}_j)\mathbf{v}_j^\top = \phi(\mathbf{q}_i)^\top S$, and likewise for the denominator. S and $\mathbf{s}$ are computed once in $O(nrd_v)$, then each of the n outputs costs $O(rd_v)$.

(b) Restricting the sums to $j \le i$ gives the partial sums $S_i, \mathbf{s}_i$, which satisfy the stated recursions with $S_0 = 0$, $\mathbf{s}_0 = \mathbf{0}$.

(c) For $\mathbf{w} \sim N(\mathbf{0}, I)$ and $\mathbf{u} \in \mathbb{R}^d$, $\mathbb{E}[e^{\mathbf{w}\cdot\mathbf{u}}] = e^{\|\mathbf{u}\|^2/2}$, by the moment generating function of $\mathbf{w}\cdot\mathbf{u} \sim N(0, \|\mathbf{u}\|^2)$. Hence

$$\mathbb{E}[\phi_{\mathbf{w}}(\mathbf{q})\phi_{\mathbf{w}}(\mathbf{k})] = e^{-\frac{1}{2}\|\mathbf{q}\|^2 - \frac{1}{2}\|\mathbf{k}\|^2}\, \mathbb{E}[e^{\mathbf{w}\cdot(\mathbf{q}+\mathbf{k})}] = e^{\frac{1}{2}(\|\mathbf{q}+\mathbf{k}\|^2 - \|\mathbf{q}\|^2 - \|\mathbf{k}\|^2)} = e^{\mathbf{q}\cdot\mathbf{k}}.$$

■

Averaging $\phi_{\mathbf{w}}$ over r independent draws of $\mathbf{w}$ gives an unbiased approximation of softmax attention in linear time (the Performer idea) [Supplement].

12 Why Transformers

The original Transformer was developed for machine translation. It turned out to scale surprisingly well with large datasets, and most large-scale models since (BERT, GPT-2, Megatron-LM, T5, Turing-NLG, with up to 17 billion parameters by early 2020) are built on it. Its weak inductive bias (Theorem 10.22: only PE carries structure) is a cost on small data and an advantage once data is plentiful.

13 Selected exercises

Motivation. Five derivations: the softmax temperature and why large logits are harmful, attention as kernel smoothing, the cost and rank of multi-head attention, the arithmetic of positional encodings, and the gradient of layer normalization.

Exercise 10.1 (Softmax temperature)

For $\mathbf{e} \in \mathbb{R}^N$ and $\beta \ge 0$ let $\mathbf{a}(\beta) = \mathrm{softmax}(\beta\mathbf{e})$ and $H(\beta) = -\sum_i a_i \log a_i$.
(a) Show that $\frac{d}{d\beta}\sum_i a_i e_i = \mathrm{Var}_{\mathbf{a}}(\mathbf{e})$, where $\mathrm{Var}_{\mathbf{a}}(\mathbf{e}) = \sum_i a_i e_i^2 - (\sum_i a_i e_i)^2$.
(b) Show that $H'(\beta) = -\beta\,\mathrm{Var}_{\mathbf{a}}(\mathbf{e}) \le 0$.
(c) Show that $\mathbf{a}(0)$ is uniform, and that as $\beta \to \infty$, $\mathbf{a}(\beta)$ converges to the uniform distribution on $\arg\max_i e_i$.
(d) Explain with Theorem 10.8 why unscaled logits $\mathbf{q}\cdot\mathbf{k}$ behave like $\beta = \sqrt{D}$ applied to unit-variance logits.

Solution. [Supplement]

(a) Let $m(\beta) = \sum_i a_i e_i$. By Theorem 10.9(a) and the chain rule, $da_i/d\beta = \sum_j J_{ij} e_j = a_i(e_i - m)$. Hence $m' = \sum_i a_i e_i (e_i - m) = \sum_i a_i e_i^2 - m^2$.

(b) $\log a_i = \beta e_i - \mathrm{LSE}(\beta \mathbf{e})$, so $H = \mathrm{LSE}(\beta \mathbf{e}) - \beta m$. Since $\frac{d}{d\beta} \mathrm{LSE}(\beta \mathbf{e}) = \sum_i a_i e_i = m$, we get $H' = m - m - \beta m' = -\beta \operatorname{Var}_{\mathbf{a}}(\mathbf{e})$, which is ≤ 0 as a variance times $-\beta \le 0$.

(c) At $\beta = 0$ all weights are e^0/N. Let $e^* = \max_i e_i$ and $K = \{i : e_i = e^*\}$. Divide numerator and denominator by $e^{\beta e^*}$: $a_i = e^{\beta(e_i - e^*)} / \sum_j e^{\beta(e_j - e^*)}$. As $\beta \to \infty$, $e^{\beta(e_j - e^*)} \to 1$ for $j \in K$ and $\to 0$ otherwise. So $a_i \to 1/|K|$ for $i \in K$ and $a_i \to 0$ otherwise.

(d) By Theorem 10.8, $\mathbf{q} \cdot \mathbf{k} = \sqrt{D} \cdot (\mathbf{q} \cdot \mathbf{k}/\sqrt{D})$, where the second factor has unit variance. So unscaled attention is the softmax of unit-variance logits at inverse temperature $\beta = \sqrt{D}$. By (b) and (c) its entropy falls toward that of a one-hot vector as D grows, and Theorem 10.9(c) shows the gradient vanishes there. Dividing by $\sqrt{D}$ keeps $\beta = 1$. ■

Exercise 10.2 (Attention as kernel smoothing)

Let keys $\mathbf{k}_1, \dots, \mathbf{k}_N \in \mathbb{R}^D$ all have the same norm, values $v_j \in \mathbb{R}$, a query $\mathbf{q}$, and $\tau > 0$. Let $y(\mathbf{q}) = \sum_j a_j v_j$ with $\mathbf{a} = \mathrm{softmax}(\mathbf{q} \cdot \mathbf{k}_j/\tau)_j$.
(a) Show that $a_j = \exp(-\|\mathbf{q} - \mathbf{k}_j\|^2/2\tau) / \sum_l \exp(-\|\mathbf{q} - \mathbf{k}_l\|^2/2\tau)$, so y is a Nadaraya–Watson estimator with a Gaussian kernel.
(b) If one key $\mathbf{k}_{j^*}$ is strictly closest to $\mathbf{q}$, show that $y(\mathbf{q}) \to v_{j^*}$ as $\tau \to 0$.
(c) If $\mathbf{q} \cdot \mathbf{k}_{j^*} - \mathbf{q} \cdot \mathbf{k}_j \ge \Delta > 0$ for all $j \ne j^*$, show that $a_{j^*} \ge 1/(1 + (N-1)e^{-\Delta/\tau})$.

Solution. [Supplement]

(a) $-\|\mathbf{q} - \mathbf{k}_j\|^2/2\tau = (\mathbf{q} \cdot \mathbf{k}_j)/\tau - (\|\mathbf{q}\|^2 + \|\mathbf{k}_j\|^2)/2\tau$. The last term is the same for every j, because the keys have equal norms, so it cancels between numerator and denominator.

(b) Strictly closest means $\mathbf{q} \cdot \mathbf{k}_{j^*}$ is the unique maximum of $\mathbf{q} \cdot \mathbf{k}_j$, by (a). By Exercise 10.1(c) with $\beta = 1/\tau \to \infty$, $\mathbf{a} \to \mathbf{e}_{j^*}$, hence $y \to v_{j^*}$.

(c) Divide by $e^{\mathbf{q} \cdot \mathbf{k}_{j^*}/\tau}$: $a_{j^*} = 1/(1 + \sum_{j \ne j^*} e^{-(\mathbf{q} \cdot \mathbf{k}_{j^*} - \mathbf{q} \cdot \mathbf{k}_j)/\tau}) \ge 1/(1 + (N-1)e^{-\Delta/\tau})$. ■

Exercise 10.3 (Multi-head attention)

Use Definition 10.19 with self-attention inputs $Q = K = V = X \in \mathbb{R}^{n \times d}$.
(a) Suppose all heads share the same W_q, W_k, so that they share one attention matrix A. Show that $\mathrm{MHA}(X) = AXW'$ for a single matrix $W' \in \mathbb{R}^{d \times d}$, so the layer is single-head attention with a different value map.
(b) Count the multiply–adds of computing the h attention matrices $QW_q^{(r)}(KW_k^{(r)})^\top$ with $d_k = d/h$, and compare with one head of size $d_k = d$.
(c) Show that each head's logit matrix $QW_q^{(r)}(KW_k^{(r)})^\top \in \mathbb{R}^{n \times n}$ has rank at most $d_k = d/h$. What does this say when $n > d/h$?

Solution. [Supplement]

(a) By Theorem 10.20, $\mathrm{MHA}(X) = \sum_r AXW_v^{(r)} W_o^{(r)} = AX(\sum_r W_v^{(r)} W_o^{(r)})$. Take $W' = \sum_r W_v^{(r)} W_o^{(r)}$.

(b) Each head multiplies $n \times \frac{d}{h}$ by $\frac{d}{h} \times n$: $n^2 \frac{d}{h}$ operations, so $n^2 d$ for all h heads. One head of size d costs $n^2 d$ too. Splitting into heads keeps the cost and gives h different attention patterns.

(c) The logit matrix is the product of an $n \times d_k$ and a $d_k \times n$ matrix, and $\mathrm{rank}(BC) \le \min(\mathrm{rank}\, B, \mathrm{rank}\, C) \le d_k$. When $n > d/h$ each head's logits are a low-rank matrix, so one head

cannot realize an arbitrary pattern of scores over all pairs of tokens. Several heads, combined by W_o, give several such patterns at the same total cost. ■

Exercise 10.4 (Arithmetic of positional encodings)

Use Definition 10.14.
(a) Show that $\mathrm{PE}(t) \neq \mathrm{PE}(s)$ for distinct integers $t \neq s$.
(b) Bit k of the binary expansion of t is periodic in t with period 2^{k+1}. Find the period of the pair $(\sin \omega_k t, \cos \omega_k t)$, and show that the periods form a geometric sequence from 2π to less than $10000 \cdot 2\pi$.
(c) A learned lookup table has $T \times d$ parameters. What does it output for a position $t \geq T$, and what does the sinusoidal encoding output?

Solution. [Supplement]

(a) Take $k = 0$, where $\omega_0 = 1$. Equality of the first pair gives $\sin t = \sin s$ and $\cos t = \cos s$, so $t - s \in 2\pi\mathbb{Z}$. A nonzero integer is not a multiple of 2π, because π is irrational. So $t = s$.

(b) The pair has period $2\pi/\omega_k = 2\pi \cdot 10000^{2k/d}$. Consecutive periods have ratio $10000^{2/d}$, so they form a geometric sequence. It starts at 2π ($k = 0$) and ends at $2\pi \cdot 10000^{(d-2)/d} < 2\pi \cdot 10000$ ($k = \frac{d}{2} - 1$). The counter's periods $2, 4, 8, \ldots$ are geometric with ratio 2 in the same way.

(c) The table has no entry for $t \geq T$, so a learned encoding cannot be used beyond the training length without extra rules. The sinusoidal encoding is defined for every t, and by Theorem 10.15(a) it relates t to $t + m$ in the same way at every position. ■

Exercise 10.5 (The gradient of layer normalization)

Take $\epsilon = 0$, $\sigma > 0$, and $\hat{\mathbf{x}} = (\mathbf{x} - \mu\mathbf{1})/\sigma$ as in Definition 10.23.
(a) Show that $\dfrac{\partial \hat{\mathbf{x}}}{\partial \mathbf{x}} = \dfrac{1}{\sigma}\Big(I - \dfrac{1}{d}\mathbf{1}\mathbf{1}^\top - \dfrac{1}{d}\hat{\mathbf{x}}\hat{\mathbf{x}}^\top\Big)$.
(b) Show that this Jacobian is $\frac{1}{\sigma}$ times the orthogonal projection onto $\{\mathbf{1}, \hat{\mathbf{x}}\}^\perp$.
(c) Deduce that the Jacobian annihilates $\mathbf{1}$ and $\mathbf{x}$, and explain this via Theorem 10.24(b).

Solution. [Supplement]

(a) Let $C = I - \frac{1}{d}\mathbf{1}\mathbf{1}^\top$ and $\mathbf{c} = C\mathbf{x} = \mathbf{x} - \mu\mathbf{1}$. Then $\sigma = \|\mathbf{c}\|/\sqrt{d}$ and $\hat{\mathbf{x}} = \sqrt{d}\,\mathbf{c}/\|\mathbf{c}\|$. Differentiating $\mathbf{c} \mapsto \mathbf{c}/\|\mathbf{c}\|$ gives $\frac{1}{\|\mathbf{c}\|}(I - \frac{\mathbf{c}\mathbf{c}^\top}{\|\mathbf{c}\|^2})$. Since $\mathbf{c}/\|\mathbf{c}\| = \hat{\mathbf{x}}/\sqrt{d}$ and $\sqrt{d}/\|\mathbf{c}\| = 1/\sigma$,

$$\frac{\partial \hat{\mathbf{x}}}{\partial \mathbf{c}} = \frac{1}{\sigma}\Big(I - \frac{1}{d}\hat{\mathbf{x}}\hat{\mathbf{x}}^\top\Big), \qquad \frac{\partial \hat{\mathbf{x}}}{\partial \mathbf{x}} = \frac{1}{\sigma}\Big(I - \frac{1}{d}\hat{\mathbf{x}}\hat{\mathbf{x}}^\top\Big)C.$$

By Theorem 10.24(a), $\hat{\mathbf{x}}^\top\mathbf{1} = 0$, so $\hat{\mathbf{x}}^\top C = \hat{\mathbf{x}}^\top$. Expanding the product gives the formula.

(b) By Theorem 10.24(a), $\mathbf{1}/\sqrt{d}$ and $\hat{\mathbf{x}}/\sqrt{d}$ are orthonormal. For orthonormal $\mathbf{u}_1, \mathbf{u}_2$, $I - \mathbf{u}_1\mathbf{u}_1^\top - \mathbf{u}_2\mathbf{u}_2^\top$ is the orthogonal projection onto $\{\mathbf{u}_1, \mathbf{u}_2\}^\perp$.

(c) The projection kills $\mathbf{1}$ and $\hat{\mathbf{x}}$, hence also $\mathbf{x} = \sigma\hat{\mathbf{x}} + \mu\mathbf{1}$. By Theorem 10.24(b), $\hat{\mathbf{x}}$ does not change along $\mathbf{x} \mapsto \mathbf{x} + b\mathbf{1}$ or $\mathbf{x} \mapsto a\mathbf{x}$, so its derivative in the directions $\mathbf{1}$ and $\mathbf{x}$ must be 0. ■